Expt. No.8

Title: *Design and implement a CNN model for Digit Recognition*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 15/09/2025

Sign: _____

Experiment 8

Aim: Design and implement a CNN model for digit recognition

Software: Google Collab

Pre Lab Question:

Explain the CNN architecture?

CNN architecture consists of convolutional, pooling, and fully connected layers for automatic feature extraction and classification.

Post Lab Question:

Explain relation between input, output and filter size?

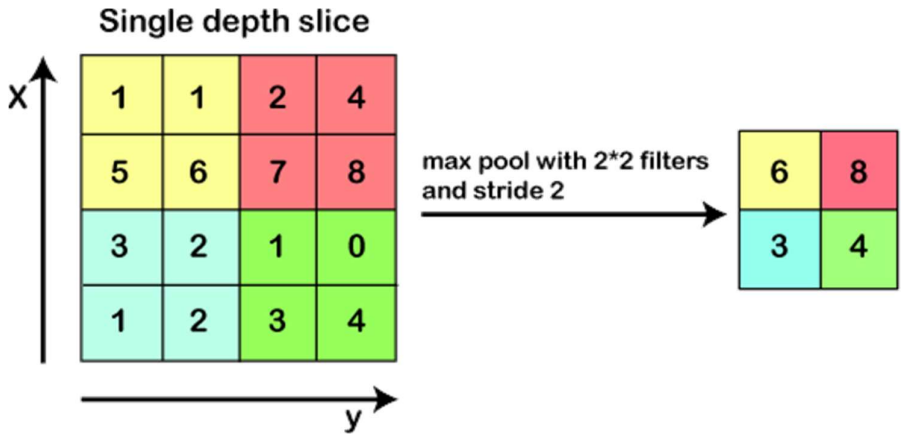
In CNNs, output size depends on input size, filter size, stride, and padding during convolution operations.

Theory: A convolutional neural network, or CNN, is a deep learning neural network sketched for processing structured arrays of data such as portrayals. CNN are very satisfactory at picking up on design in the input image, such as lines, gradients, circles, or even eyes and faces. This characteristic that makes convolutional neural network so robust for computer vision. CNN can run directly on a underdone image and do not need any pre-processing convolutional neural network is a feed forward neural network, seldom with up to 20.The strength of a convolutional neural network comes from a particular kind of layer called the convolutional layer. CNN contains many convolutional layers assembled on top of each other, each one competent of recognizing more sophisticated shapes.

CNN architecture

Generally, a Convolutional Neural Network has three layers, which are as follows;

- **Input:** If the image consists of 32 widths, 32 height encompassing three R, G, B channels, then it will hold the raw pixel([32x32x3]) values of an image.
- **Convolution:** It computes the output of those neurons, which are associated with input's local regions, such that each neuron will calculate a dot product in between weights and a small region to which they are actually linked to in the input volume. For example, if we choose to incorporate 12 filters, then it will result in a volume of [32x32x12].
- **ReLU Layer:** It is specially used to apply an activation function elementwise, like as max (0, x) thresholding at zero. It results in ([32x32x12]), which relates to an unchanged size of the volume.
- **Pooling:** This layer is used to perform a down sampling operation along the spatial dimensions (width, height) that results in [16x16x12] volume.


Single depth slice
X
y
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2*2 filters
and stride 2
6 8
3 4

Program & Output:

```
from keras.layers import Conv2D, Input, MaxPooling2D, Flatten, Dense
from keras.models import Sequential
from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

model = Sequential()

model.add(Input(shape=(28,28,1)))

model.add(Conv2D(32, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())

model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10, batch_size=35, validation_data=(X_test, y_test))
```

```
Epoch 1/10
1715/1715 ━━━━ 12s 4ms/step - accuracy: 0.9002 - loss: 0.3207 - val_accuracy: 0.9856 -
val_loss: 0.0457

Epoch 2/10
1715/1715 ━━━━ 16s 4ms/step - accuracy: 0.9848 - loss: 0.0482 - val_accuracy: 0.9887 -
val_loss: 0.0357

Epoch 3/10
1715/1715 ━━━━ 10s 4ms/step - accuracy: 0.9902 - loss: 0.0307 - val_accuracy: 0.9883 -
val_loss: 0.0346

Epoch 4/10
1715/1715 ━━━━ 10s 3ms/step - accuracy: 0.9927 - loss: 0.0230 - val_accuracy: 0.9899 -
val_loss: 0.0295
```

Epoch 5/10
1715/1715 ——— 6s 3ms/step - accuracy: 0.9947 - loss: 0.0165 - val_accuracy: 0.9901 - val_loss: 0.0374

Epoch 6/10
1715/1715 ——— 10s 3ms/step - accuracy: 0.9955 - loss: 0.0133 - val_accuracy: 0.9916 - val_loss: 0.0306

Epoch 7/10
1715/1715 ——— 5s 3ms/step - accuracy: 0.9972 - loss: 0.0091 - val_accuracy: 0.9918 - val_loss: 0.0316

Epoch 8/10
1715/1715 ——— 10s 3ms/step - accuracy: 0.9970 - loss: 0.0089 - val_accuracy: 0.9909 - val_loss: 0.0330

Epoch 9/10
1715/1715 ——— 6s 3ms/step - accuracy: 0.9975 - loss: 0.0066 - val_accuracy: 0.9899 - val_loss: 0.0375

Epoch 10/10
1715/1715 ——— 6s 3ms/step - accuracy: 0.9979 - loss: 0.0061 - val_accuracy: 0.9902 - val_loss: 0.0425

```
import numpy as np
sample = np.expand_dims(X_test[7], axis=0)
# sample = np.expand_dims(sample, axis=-1)
sample.shape

np.argmax(model.predict(sample))
```

OUTPUT:

```
np.int64(9)

y_test[7]
np.uint8(9)
```

Conclusion:

Designing and implementing a CNN model for digit recognition enables automatic feature extraction and accurate classification of handwritten digits. This approach leverages convolutional layers to capture spatial patterns, significantly improving recognition performance over traditional methods.

CO's Covered:

CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.
CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.